

Cours de Programmation en C – EI-2I

Travaux Dirigés

TD6

Exercice 1 – Nombres complexes

On cherche à représenter des nombres complexes (donc de la forme $a + ib$) et à calculer avec.

1. Définir un type structuré permettant de représenter un nombre complexe ;
2. Écrire une procédure permettant d'afficher un nombre complexe (par exemple sous la forme $3.65 + 0.12i$) ;
3. Écrire une fonction permettant de calculer le module d'un nombre complexe ;
4. Écrire une procédure permettant de calculer la somme de deux nombres complexes ;
5. Écrire une procédure permettant de calculer le produit de deux nombres complexes ;
6. Écrire un programme principal, utilisant ces procédures, et affichant le résultat de $(1 - i) * (3 - 0.5i) + 2i$ et son module.

Solution:

```
#include <stdlib.h>
#include <stdio.h>
#include <math.h>

typedef struct
{
    double reel;
    double imag;
} t_complexe;

/* affiche un complexe */
void affiche( t_complexe c)
{
    printf( "%f+%fi", c.reel, c.imag);
}

/* calcule le module d'un complexe */
double module( t_complexe c)
{
    return sqrt( c.reel*c.reel + c.imag*c.imag);
}

/* additionne deux complexes
le résultat est mis dans la variable pointée par pRes */
void add( t_complexe* pRes, t_complexe op1, t_complexe op2)
{
    pRes->reel = op1.reel + op2.reel;
    pRes->imag = op1.imag + op2.imag;
}

/* multiplie deux complexes
le résultat est mis dans la variable pointée par pRes */
void mul( t_complexe* pRes, t_complexe op1, t_complexe op2)
{
    pRes->reel = op1.reel*op2.reel - op1.imag*op2.imag;
    pRes->imag = op1.imag*op2.reel + op1.reel*op2.imag;
}

int main()
{
    t_complexe x,y,z;
    t_complexe res1,res2;

    /* définition de nos complexes */
    x.reel=1; x.imag=-1;
    y.reel=3; y.imag=-0.5;
    z.reel=0; z.imag=2;

    /* calcul et affichage */
    mul( &res1, x, y);
    add( &res2, res1, z);
    printf( "|");
    affiche( res2);
    printf( " |=%f", module(res2) );

    return EXIT_SUCCESS;
}
```

Exercice 2 – Structures de personne

1. Définir un type `t_chaine` pour des tableaux de 40 caractères.
2. Définir un type `t_date` pour définir une date

3. Définir un type `t_personne` pour une personne, avec son nom, son prénom, sa date de naissance, son âge, et son adresse (tableau de 3 chaînes de caractères).

4. Définir une procédure permettant d'afficher une personne passée en paramètre. Faut-il mieux passer un `t_personne` ou un `t_personne*` ?

5. Écrire une procédure permettant de lire les informations d'une personne dans un fichier ouvert (où les données sont écrites dans un ordre particulier)

Solution:

```
#include <stdio.h>
#include <stdlib.h>

typedef char t_chaine[40];

typedef struct
{
    int jour;
    int mois;
    int annee;
} t_date;

typedef struct
{
    t_chaine nom;
    t_chaine prenom;
    int age;
    t_date naissance;
    t_chaine adresse[3];
} t_personne;

void affichePersonne( t_personne p)
{
    int i;
    printf( "Nom_: %s\n", p.nom);
    printf( "Prenom_: %s\n", p.prenom);
    printf( "Date_de_naissance_: %d/%d/%d\n", p.naissance.jour, p.naissance.mois, p
        .naissance.annee);
    printf( "Adresse_: ");
    for( i=0; i<3; i++)
        printf( "%s\n", p.adresse[i]);
}

void lecturePersonne( FILE* fichier, t_personne* pPers)
{
    int i;
    fscanf( fichier, "%s", pPers->nom);
    fscanf( fichier, "%s", pPers->prenom);
    fscanf( fichier, "%d_%d_%d", &(pPers->naissance.jour), &(pPers->naissance.mois)
        , &(pPers->naissance.annee));
    for( i=0; i<3; i++)
        fscanf( fichier, "%s", pPers->adresse[i]);
}

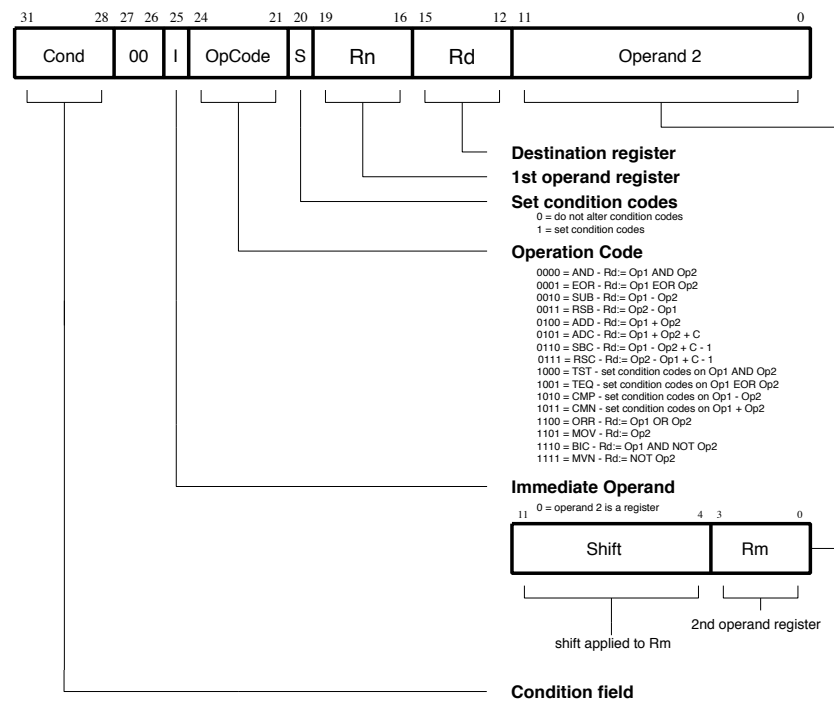
int main()
{
    FILE* fichier = fopen( "pers.txt", "r");
    t_personne carnet[3];
    int i;

    if (fichier)
    {
        for( i=0; i<3; i++)
        {
            lecturePersonne( fichier, &(carnet[i]) );
            affichePersonne( carnet[i] );
        }
    }

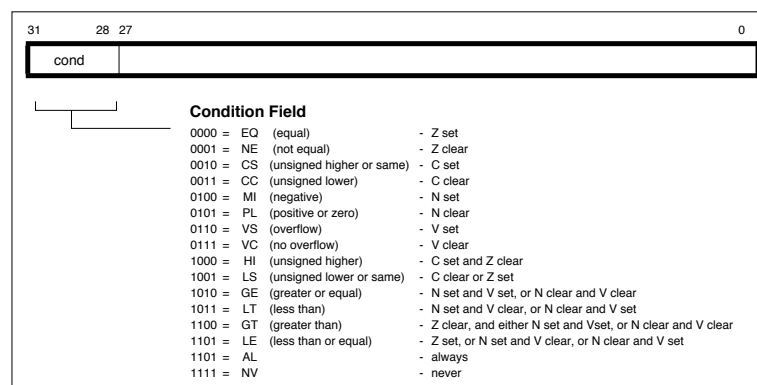
    return EXIT_SUCCESS;
}
```

Exercice 3 – Jeu d’instruction ARM

L’ARM7 est un processeur RISC 32 bits. Une partie de son jeu d’instruction est défini de la manière suivante :



où le champ `Cond` est défini par



Définir les types `t_Cond`, `t_Opcode` et `t_instruction` pour les champs `Cond`, `OpCode` et l’instruction, respectivement.

Solution:

```
typedef enum
{
    EQ, NE, CS, CC, MI, PL, VS, VC, HI, LS, GE, LT, GT, LE, AL, NV
} t_cond;

typedef enum
{
    AND, EOR, SUB, RSB, ADD, ADC, SBC, RSC, TST, TEQ, CMP, CMN, ORR, MOV, BIC, MVN
} t_opCode;

typedef struct
{
    unsigned int Rm : 4;
    signed int Shift : 8;
    unsigned int Rd : 4;
    unsigned int Rn : 4;
    unsigned int S : 1;
    t_opCode OpCode : 4;
    unsigned int I : 1;
    unsigned int : 2;
    t_cond Cond : 4;
} t_instruction;
```